

Plan → mission → report

The loop runs itself: plan in the app, the robot flies and reports, models build automatically. Humans plan and review. Today we run it live.

YOU LEAVE WITH

The full loop run live end to end, the two ROS topics your side integrates with, and a sandbox to practise in

BRING

A laptop with Chrome and your Fusion login



The hour

09:15 5' The loop and the one rule: **CDF is the only integration point**

09:20 25' **Live demo:** plan in the app → topics on the robot → mission + findings → automatic upload → 3D models → review

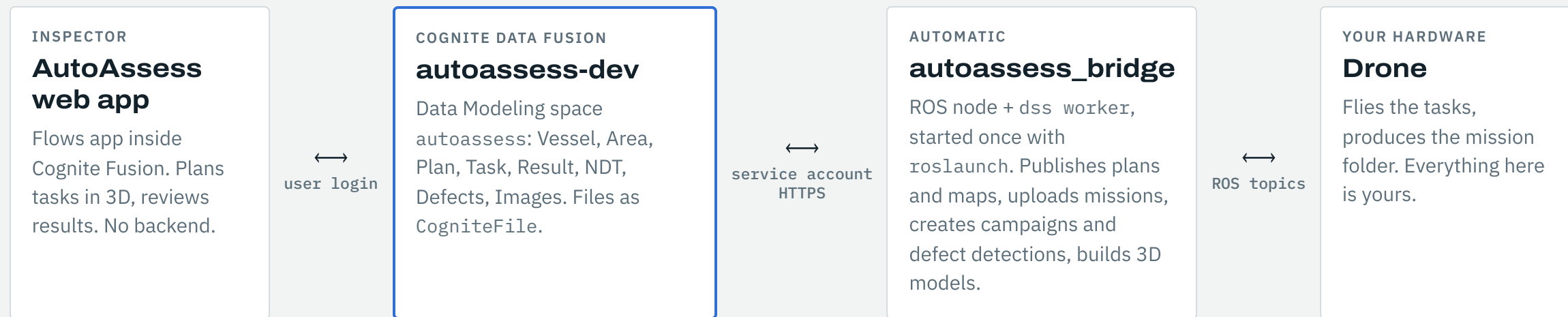
09:45 8' Your two interfaces: `plan.json` in, findings JSON out (+ mission files)

09:53 7' Hardware and environment · app deployment

10:00 12' Hands-on in the sandbox (works without a robot)

10:12 3' Open questions we need your answer on

Four parts, one integration point



The web app and the ground station never talk to each other. Everything goes through CDF. Your integration surface is **two ROS topics**: the plan arrives on `/autoassess/plan`, findings leave on `/autoassess/findings`. Everything in between is automatic.

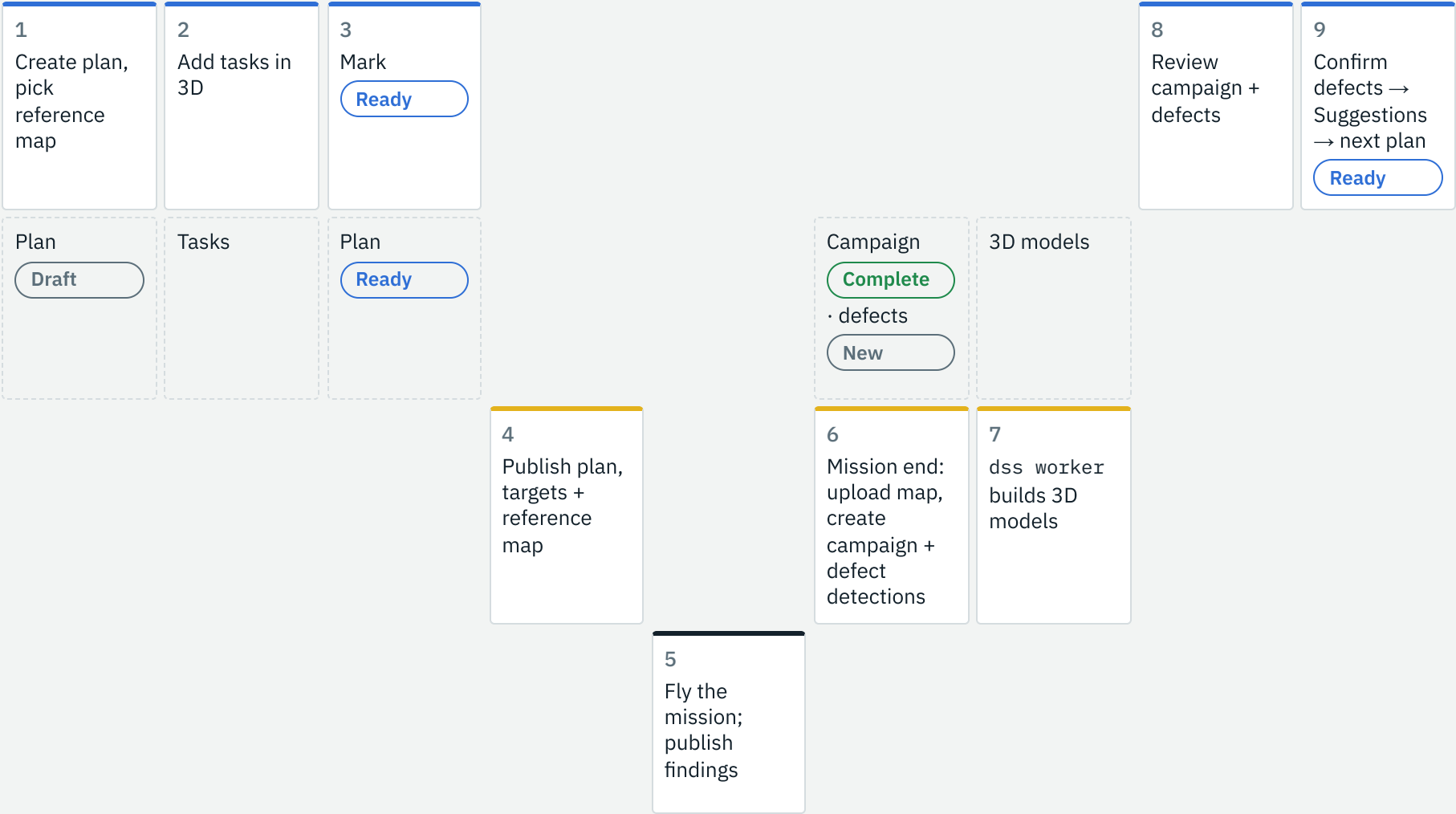
Mission lifecycle

Inspector
web app

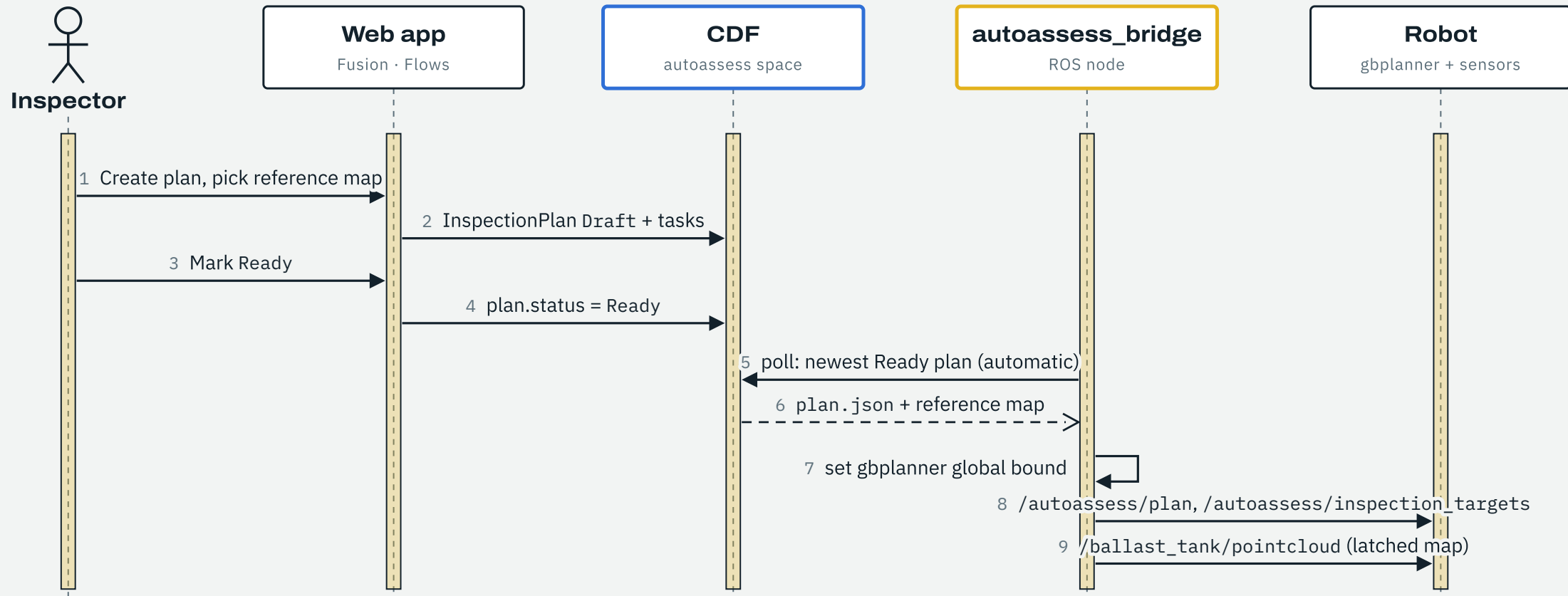
CDF
autoassess space

Bridge
automatic (ROS)

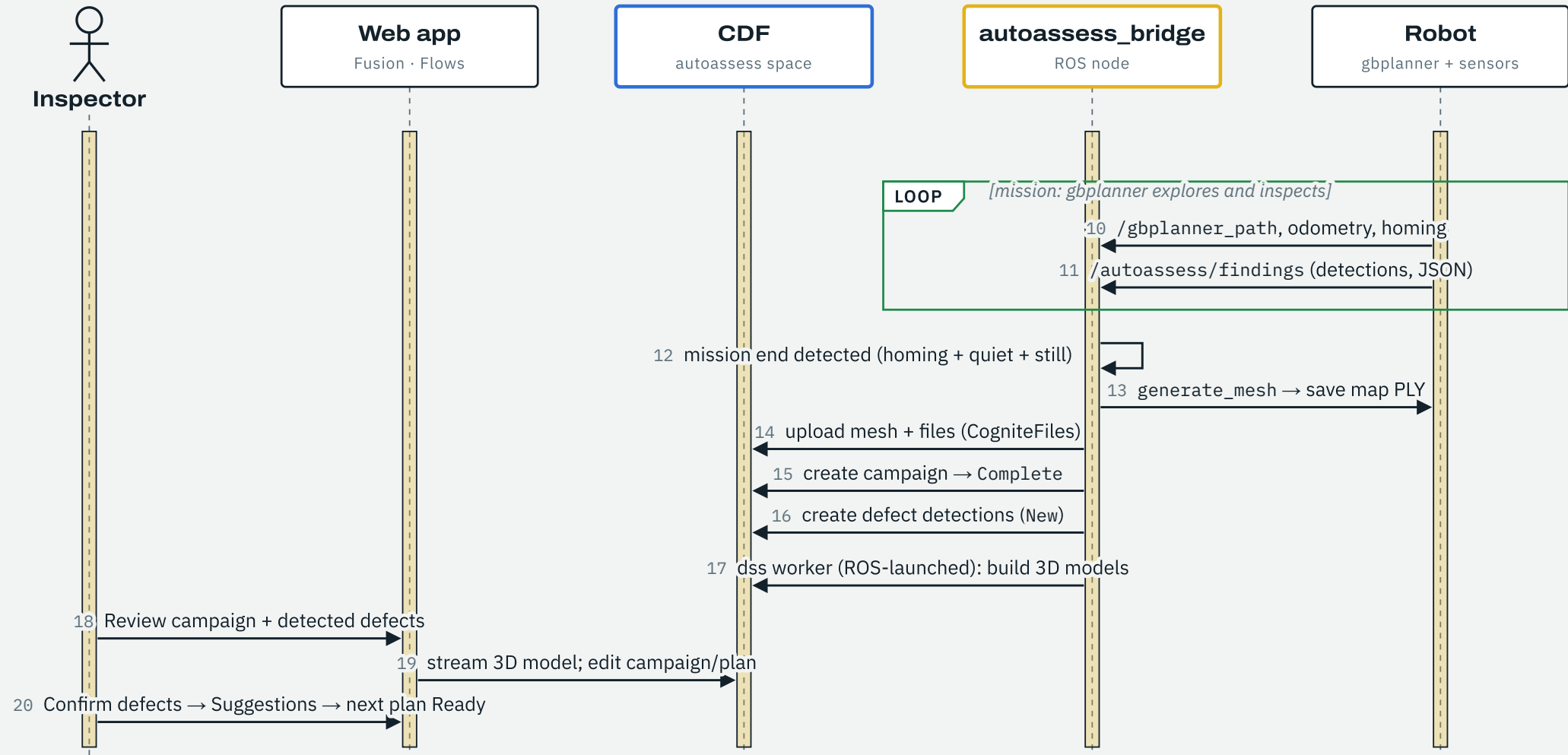
Robot
gbplanner + your stack



Sequence 1/2: plan → robot, all automatic

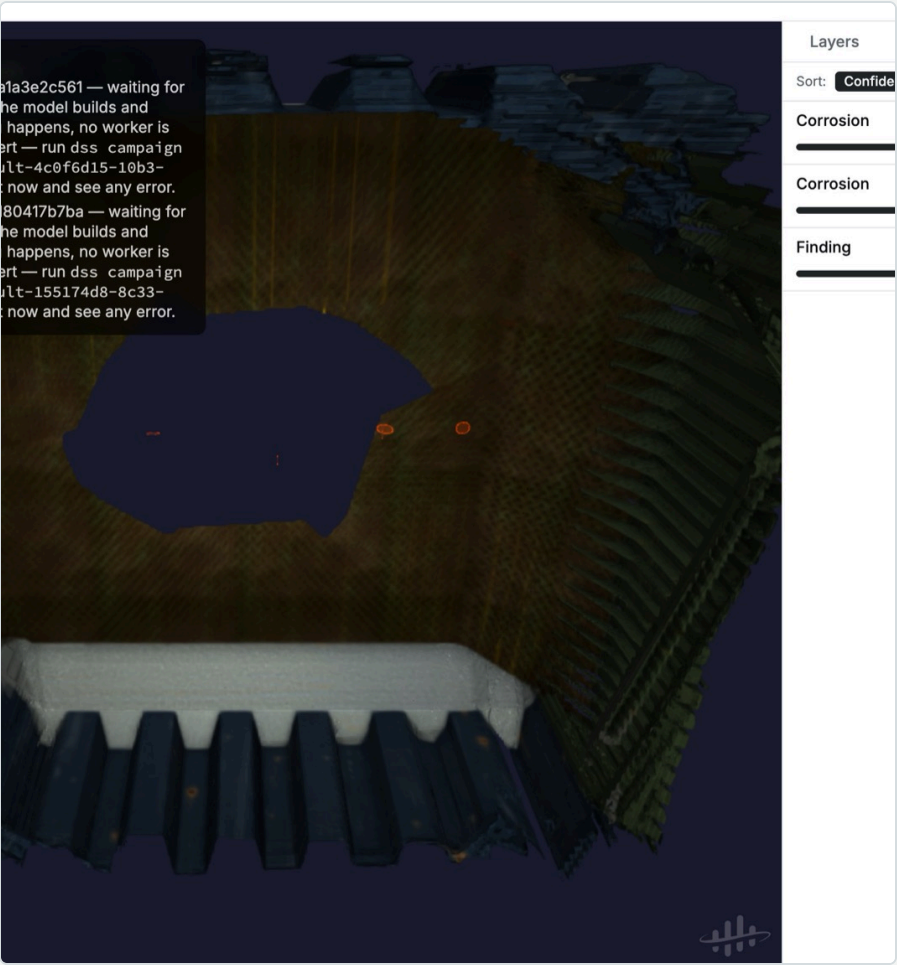


Sequence 2/2: mission → findings → models → review



The live demo, step by step

STEP	WHO / WHAT	WATCH
1	Mark the plan Ready	Plans tab
2	Bridge follows it (~7 s)	<code>rostopic echo /autoassess/plan_id</code>
3	Targets + reference map appear	<code>/autoassess/inspection_targets</code> , <code>/ballast_tank/pointcloud</code>
4	Mission flies (gbplanner / stub)	<code>/gbplanner_path</code> , <code>rviz</code>
5	Detections published	<code>rostopic pub /autoassess/findings ...</code>
6	Mission end → auto upload	<code>/autoassess/upload_status</code> → complete
7	Campaign + defects (New) in CDF	Viewer: Layers + Defects tab
8	<code>dss worker</code> builds models	Viewer: notice → streamed mesh
9	Confirm defect → Suggestions → plan Ready	Bridge switches: loop closed



No robot? The same loop runs against a stub gbplanner in Docker: `autoassess_bridge/e2e/full_loop.sh`.

Hardware and environment

Ground station INSTALL ONCE

- ROS 1 Noetic (Python 3.8) for `autoassess_bridge`
- `uv` + Python 3.12 checkout of the AutoAssess SDK for `dss_worker` (one `roslaunch autoassess_full.launch` starts both)
- Outbound HTTPS (443) to `<cluster>.cognitedata.com` and the login provider
- CDF service-account credentials in a git-ignored `.env`; uploads need write access
- Disk for maps and meshes: a single mesh can be 100–270 MB

Drone and sensors DERIVED

- Localise in the **reference map frame** of the plan's `mapExternalId`
- 3D capture: mesh (PLY, per-vertex RGB) and/or point cloud (PCD)
- RGB camera with known intrinsics and body→camera extrinsic
- Pose log at image timestamps (TUM format)
- UT thickness gauge for `ndt_thickness` tasks

Accuracy and reviewers

FROM
THE PRD

- **KO-5:** < 1 cm relative accuracy of defect/NDT registration
- **NFR-04:** always cloud-connected at upload time, no offline mode
- **NFR-05:** reviewers need Chrome/Edge/Firefox with WebGL 2
- **KO-7:** demonstrated on bulk carrier, container ship and tanker

TBD PARTNER

Onboard compute · link between drone and ground station (tether, Wi-Fi, SD card) · positioning accuracy achieved in a tank · who converts sensor output to PLY/PCD/TUM

Input: plan.json

{

Tasks are an unordered set

The drone decides the order. The sandbox uses nearest-neighbour as an example.

Two kinds of task

element: inspect a structural element at its centre point. **region:** a surface point with outward normal and radius.

Findings go the other way

Publish detections as JSON on /autoassess/findings:

```
{"id","x","y","z","nx","ny","nz","radius","inspection_type","class","confidence","description"}
```

— only x,y,z required. At mission end they become defect detections on the campaign; confirming them turns

Suggestions into plan tasks.

}

User interactions

INSPECTOR · BEFORE

Plans the mission

Fusion → AutoAssess

1. Open vessel → area → 3D viewer
2. Create a plan, choose the reference map
3. Click elements or surfaces, add as tasks
4. Mark the plan Ready

BRIDGE + ROBOT · AUTOMATIC

Runs itself

`roslaunch autoassess_full.launch` (once)

1. Bridge publishes the newest Ready plan, targets and reference map
2. gbplanner flies; your stack publishes findings on `/autoassess/findings`
3. Mission end detected → map uploaded, campaign created Complete
4. Findings become defect detections (New); `dss_worker` builds the 3D models

REVIEWER · AFTER

Reviews the result

Fusion → AutoAssess

1. Toggle mesh, point clouds, NDT, images per campaign
2. Campaign report: stats, NDT table, images
3. Triage defects
4. Accept suggestions → next Draft plan

Output: what the mission leaves behind (uploaded automatically)

FILE	BECOMES IN CDF	FORMAT NOTES
*.ply	Mesh on the campaign, then a streamed 3D model	Binary or ASCII; per-vertex RGB (camera colour), optional face colours (segments)
*.pcd	Point cloud on the campaign, with a label	PCL PCD; a label field colours points
*.csv	NdtMeasurement nodes	timestamp,thickness,x,y,z: timestamp in ns , thickness in metres
ssg.yaml	StructuralElement nodes on the area	Scene graph: manhole, longitudinal, wall, compartment
metrics.yaml	CampaignMetric nodes	Decimal or percentage values
tum/	DroneImage nodes + PNG files	rgb.txt, groundtruth.txt (tx ty tz qx qy qz qw), sensor.yaml

All coordinates in the reference map frame. At mission end the bridge uploads the gbplanner map + the mission_dir files, creates the campaign (Complete) and a findings (Draft) plan. CSV/ssg/metrics and images still go through dss campaign upload for now. Per-task mission results are not reported back — only the campaign links to the plan.

Reality check: how do tasks reach the drone?

Confined-space inspection drones such as Flyability Elios 3 and ScoutDI Scout 137 are flown by a pilot. We found no public API for uploading waypoints to either.

- Elios 3 data goes to Flyability Cloud, which offers REST APIs for the recorded data.
- Scout 137 tags data to its position on a 3D map and uploads to Scout Portal after flight.
- So `plan.json` may be a **pilot checklist** plus a post-flight import, not an autopilot program. Either way the bridge publishes the plan and reference map, and the post-flight upload is automatic.

Which one is your drone?

A • Autonomous

Your software reads `plan.json` and calls something like `goto(pose) / inspect()`. The sandbox's `sim_drone` is shaped for this.

B • Pilot-flown

Show tasks to the pilot, then convert the vendor export (cloud API, SD card) into the mission folder before `dss campaign upload`.

PLAN

FLY

UPLOAD

REVIEW

10:00

The Drone Sandbox

Python runs in your browser (Pyodide). No install, no credentials.

- uidss: the **same API** as the real ground-station SDK, reading the real project, read-only
- `sim_drone = SimDrone()`: a simulated drone you drive with poses
- Task panel shows each task's state during the flight
- Marking the plan Complete is **simulated**. Nothing is written to CDF.

OPEN IT

<https://cog-autoassess.fusion.cognite.com/autoassess-dev/flows-apps/app/autoassess-drone-sandbox?cluster=westeurope-1.cognitedata.com&customAppVersion=0.0.1&workspace=flows>

AutoAssess Drone Sandbox
Python in your browser with the uidss SDK: pick up inspection plans, check them, send off a simulated drone.

CDF project autoassess-dev (read-only) 16 plans · 7 areas Reload Python

3. Fly the mission Run Stop Reset

```

1 """3. Fly the mission - pick up the newest Ready plan, fly it with the s
2 import json
3 from pathlib import Path
4
5 from dss_sandbox import SimDrone
6 from uidss import UidssClient
7
8 client = UidssClient.from_env()
9
10 # 1. Newest Ready plan that has tasks, across all vessels/areas.
11 candidates = [
12     (area, p)
13     for vessel in client.vessels.list()
14     for area in client.areas.list(vessel.space, vessel.external_id)
15     for p in client.plans.list(area.space, area.external_id)
16     if p.status == "Ready"
17 ]
18 counts = client.plans.count_tasks([p.external_id for _, p in candidates])
19 candidates = [(area, p) for area, p in candidates if counts[p.external_id]
20 ]
21 if not candidates:
22     raise SystemExit("No Ready plan with tasks found. Add tasks to a plan
23 area, plan_meta = max(candidates, key=lambda c: c[1].created_time)

```

Console Clear

```

t= 14.5s Arrived at (2.91, 0.29, 2.15), yaw -9°
t= 17.5s Inspected task-0853c1cd-457e-4835-b221-8f82393acb16 (visual, 3 s)
t= 17.5s Returning home
t= 28.9s Landed

Mission plan-8018715f-b235-411c-b850-a2185c1bf9e5: 2/2 tasks inspected, 11.5 m flown in 29 s (simulated)
simulated: plan plan-8018715f-b235-411c-b850-a2185c1bf9e5 → Complete (not written to CDF)

✓ Finished in 44 ms

```

Drone simulator · plan-8018715f-b235-411c-b850-a2185c1bf9e5
Test Area · speed 0.5 m/s · battery 900 s
TOP VIEW (X-Y)

SIDE VIEW (X-Z)

1 m

● Region task ● Element task ● Inspected ● Skipped — Drone path ■ Structural

Replay Restart Speed 10x 28

Tasks · plan-8018715f-b235-411c-b850-a2185c1bf9e5 simulated

task-0853c1cd-457e-4835-b221-8f82393acb16	region · visual	inspected at
task-5542b775-25a1-4fdb-a197-78e614c7762a	region · visual	inspected at

Mission summary

Tasks inspected	2 / 2
Distance	11.5 m
Simulated time	29 s
Skipped	0

Mission log

0.0s	Take-off from (-1.63, 0.18, -0.14)
10.3s	Arrived at (2.34, 0.25, 2.16), yaw -72°
13.3s	Inspected task-5542b775-25a1-4fdb-a197-78e614c7762a (visual, 3 s)
14.5s	Arrived at (2.91, 0.29, 2.15), yaw -9°
17.5s	Inspected task-0853c1cd-457e-4835-b221-8f82393acb16 (visual, 3 s)
17.5s	Returning home
28.9s	Landed

Sandbox: what the bridge does at step 4 — fetch and check the plan

```
from uidss import UidssClient

client = UidssClient.from_env()
for vessel in client.vessels.list():
    for area in client.areas.list(vessel.space, vessel.external_id):
        for p in client.plans.list(area.space, area.external_id):
            print(p.status, p.name, p.external_id)

# exactly what 'dss plan download' writes
client.plans.download(p.space, p.external_id,
                      area.name, Path("plans/p.json"))

# your checks: pose present, inside the area,
# unit normal, reachable...
```

Console

[Clear](#)

```
[FAIL (3)] BWT 3P / Draft - needs review [Draft] {'element/visual': 2,
'region/visual': 1, 'region/ndt_thickness': 1}
  - demo-plan-draft-task-1: no pose
  - demo-plan-draft-task-2: [14.5, 0.2, 1.0] is outside the area bounds
  - demo-plan-draft-task-3: normalVector missing or not unit length
[PASS] BWT 3P / Tank 3 follow-up [Ready] {'element/visual': 4,
'element/ndt_thickness': 1, 'region/ndt_thickness': 2, 'region/visual': 1}
[PASS] BWT 3P / NDT sweep - port side [Ready] {'region/ndt_thickness': 6}
[PASS] Cargo Hold 2 / Hold 2 visual [Complete] {'element/visual': 3}
```

✓ Finished in 22 ms

Examples 1 and 2. On the real project, the verify example flags a Draft task outside its area bounds.

Sandbox: the mission (step 5), flown with poses

```
from dss_sandbox import (SimDrone, pose_for_task,
                        BatteryLowError)

sim_drone = SimDrone(speed_mps=0.5, max_flight_time_s=900)
issues = sim_drone.load_plan(plan)    # pre-flight checks

sim_drone.takeoff(height_m=1.0)
for task in plan["tasks"]:            # you pick the order
    # Pose(x, y, z, roll, pitch, yaw), map frame
    pose = pose_for_task(task, standoff_m=0.8,
                          approach_from=sim_drone.pose)

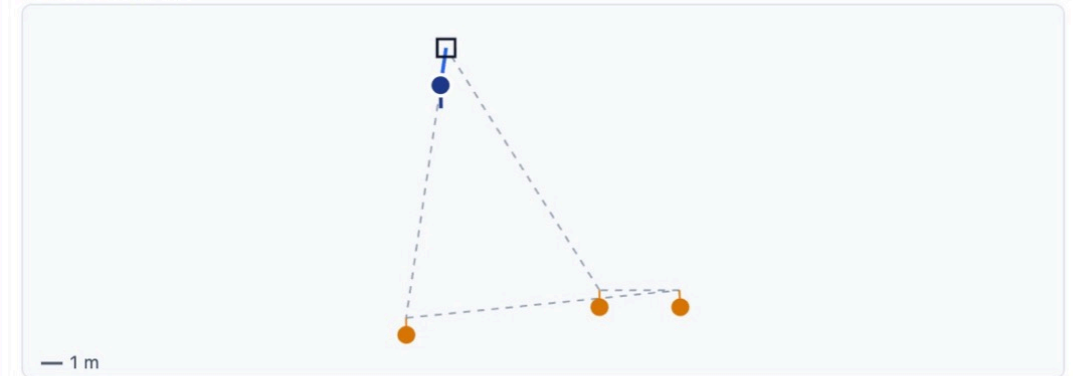
    try:
        sim_drone.goto(pose)
        sim_drone.inspect(task["id"])
    except BatteryLowError:
        break
sim_drone.return_home()
sim_drone.land()
print(sim_drone.report().summary())
```

Drone simulator · plan-9090c575-8ea3-4464-963b-727468be1cb8

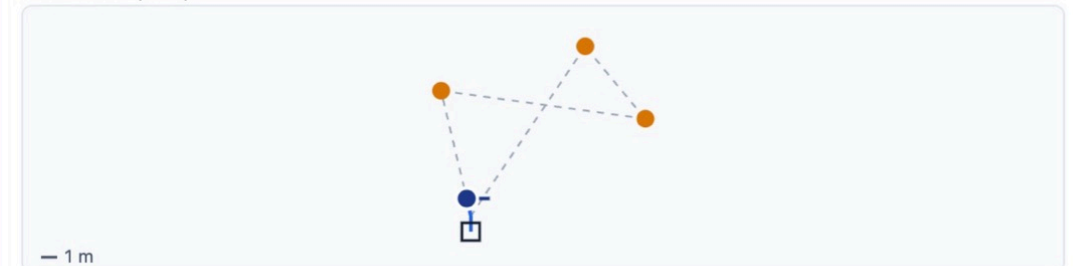
● Flying

Simulated Area · speed 0.5 m/s · battery 900 s

TOP VIEW (X-Y)



SIDE VIEW (X-Z)



● Region task ● Element task ● Inspected ● Skipped — Drone path ■ Structural element

Pause

Restart

Speed

10x

v

6.2 / 115 s

The task panel moves each task through pending → en route → inspecting → inspected (or skipped, with the reason).

Sandbox: mission end (step 6) — upload and close

Ready → mission → Complete

After landing, the example calls `client.plans.update_status(..., "Complete")`. In the sandbox this only changes the local copy, and the console says *not written to CDF*.

On the ground station, `dss campaign upload` runs first: the mission folder becomes a campaign, and then the campaign and the plan are marked Complete.

Tasks have no status in the data model. Per-task progress lives in the drone, and in the sandbox's task panel.



What you implement, what the sandbox fakes

CALL	ON YOUR DRONE	IN THE SANDBOX
load_plan(plan)	Parse plan.json, run your own pre-flight checks	Checks pose, bounds, normals; returns issues
takeoff(height_m)	Arm and climb	Climbs 1 m from the take-off point
goto(Pose)	Fly to x, y, z in the map frame, hold roll/pitch/yaw	Straight line at constant speed; battery check
inspect(task_id)	Capture images, take the UT reading, log the pose	Hover 3 s visual / 8 s NDT; task → inspected
return_home(), land()	Return and land; write the mission folder	Flies home, lands, builds the report
on_event(cb), report()	Stream progress to the operator	Events, task panel, mission summary
plans.update_status(...)	Done automatically by autoassess_bridge at mission end (campaign; the plan stays for the inspector)	Simulated; nothing written to CDF

Hands-on (5 min): open example 4, change the order or the stand-off, and make the drone skip a task on purpose. Then write the same loop against your own drone's SDK.

App deployment: how a viewer version reaches users

1 · Deploy a draft

`npx @cognite/cli apps deploy` uploads the build as a **DRAFT** of the version in `app.json` (`versionTag`). Drafts are invisible to users and open only via a `customAppVersion=<tag>` link — that's how our PRs link test builds.

2 · Sign

`apps sign`: a developer signature is required before publishing (dev projects need no certification).

3 · Publish · 4 · Activate

`apps publish` makes the version visible; `apps activate` makes it the default for users. Every step is explicit, so a bad build never reaches inspectors by accident.

- **Today:** the Reveal viewer (0.1.0-alpha.1) and the Drone Sandbox (0.0.1) are **drafts**; the active viewer version is untouched.
- **Login once:** `npx @cognite/cli auth login`; deploys then run non-interactively.
- **Quirks we hit:** stale `.cognite-bundles/` packages get re-uploaded unless deleted; App Hosting rejects archive files (`.zip/.bin`) — the sandbox embeds Python's `stdlib` in its worker because of this.
- **Published versions are immutable** — bump `versionTag` per release; drafts may be overwritten.

We need your answers

- **Findings topic:** is JSON on /autoassess/findings the right interface for your detection stack?
- **Mission end:** could gbplanner publish a latched mission_complete? Today the bridge guesses (homing + quiet + still).
- **Global bound:** with use_z_val=true the upper z limit is never applied — bug? And should the bridge grow a box that excludes the robot's position?
- **Frames & types:** is world the right default frame, and PoseArray the right type for targets?
- **plan.json / OQ-5:** does the task shape fit what your robot consumes?

TUTORIAL

docs/tutorial/ in the repo

GROUND-STATION SDK

sdk/ • `uv run dss --help`

SANDBOX

<https://cog-autoassess.fusion.cognite.com/autoassess-dev/flows-apps/app/autoassess-drone-sandbox?cluster=westeurope-1.cognitedata.com&customAppVersion=0.0.1&workspace=flows>